AI Based Age Prediction and Age Progression System

Using Face Recognition & Deep Learning

Second Review Document · March 2026

Project Title	AI Based Age Prediction and Age Progression System Using Face Recognition & Deep Learning
Review Stage	Second Review
Date	March 2026
Platform Name	AgeVision
Presented by	NEERAJ BHUVAN M (Reg No: 67224100038, Roll No: 2435MCA0011)
Guided by	Dr. P. GEETHA — Centre for Distance Education, Anna University, Chennai
Tech Stack	Angular 19 · Django 5.2 · MongoDB · PyTorch · TensorFlow · OpenCV · MiVOLO · SAM GAN

Table of Contents

1. Overall System Design
2. Dataset (Database Schema)
3. Modules Completed
4. Pseudo Code for Incomplete Modules
5. Test Cases
6. Deviations and Justifications

1. Overall System Design

1.1 System Architecture

AgeVision follows a modern four-tier client-server architecture for scalability and high-performance AI inference. The system separates concerns across Presentation, API Gateway, AI/ML Business Logic, and Data Persistence layers.

Architecture Layers

Layer	Technology	Details
Client Layer	Angular 19 Frontend	TypeScript 5.7 · Bootstrap 5.3 · Chart.js · RxJS 7.8 · ng-bootstrap 20
API Layer	Django 5.2 + DRF 3.16 (Port 8000)	Auth, Predict, Progress, History, Analytics, Settings modules
AI / ML Layer	PyTorch + TensorFlow	MiVOLO v2, InsightFace, SAM GAN, Fast-AgingGAN, DeepFace, ViT Emotion
Database Layer	MongoDB 4.6 + SQLite	Primary NoSQL data store + Django auth/sessions
External Services	HuggingFace Hub	Auto-download for MiVOLO model checkpoints

1.2 Project Structure

The project is organised as a two-application architecture:

agevision-frontend/ — Angular 19 Frontend

- pages/auth/ — Login, Register, Forgot Password components
- pages/dashboard/ — Dashboard with statistics and overview
- pages/predict/ — Age prediction (upload + camera modes)
- pages/progress/ — Age progression with GAN model selection
- pages/history/ — Prediction and progression history
- pages/analytics/ — Charts and performance analytics
- pages/settings/ — User preferences and configuration
- shared/ — Sidebar, Topbar, Logo, Dropzone components
- services/ — API, Auth, Notification, Theme, Timezone services

agevision_backend/ — Django REST API Backend

- views/ — Auth, Predict, Progress, History, Analytics, Settings views
- age_predictor.py — Ensemble age prediction interface
- mivolo_predictor.py — MiVOLO v2 Vision Transformer wrapper
- insightface_predictor.py — InsightFace RetinaFace + ArcFace fallback
- emotion_detector.py — ViT-based facial emotion classifier
- gan_progression.py — GAN-based age progression pipeline
- sam/ — SAM (Style-based Age Manipulation) with pSp + StyleGAN2

- fast_aging/ — Fast-AgingGAN CycleGAN model
- mongodb.py — MongoDB connection and data managers
- checkpoints/ — Pre-trained model weights (~4.65 GB total)

1.3 Data Flow

Age Prediction Flow

Login → Upload Image / Capture Camera → Face Detection (YOLOv8 / RetinaFace) → Face Preprocessing (crop, resize 224×224, normalize) → CNN Age Estimation (MiVOLO / InsightFace ensemble) → Gender + Emotion Detection → Bounding Box Visualization → Store in MongoDB → Display Results

Age Progression Flow

Login → Upload Image / Capture Camera → Face Detection → Current Age Estimation → Select Target Age (5–80) → Select GAN Model (SAM / Fast-AgingGAN) → GAN Inference (pSp Encoder → Age Condition → StyleGAN2 Decoder) → Post-Processing → Side-by-Side Comparison → Store in MongoDB → Display Results

1.4 Technology Stack

Layer	Technology	Version	Purpose
Frontend Framework	Angular 19	—	Standalone component SPA
UI Library	Bootstrap + ng-bootstrap	5.3 / 20	Responsive UI components
Charts	Chart.js + ng2-charts	4.5 / 8	Analytics visualization
Language (Frontend)	TypeScript	5.7	Type-safe development
Backend Framework	Django + DRF	5.2 / 3.16	REST API server
Authentication	JWT (SimpleJWT)	5.5	Stateless token-based auth
Primary Database	MongoDB	4.6	NoSQL document store
Auth Database	SQLite	—	Django auth & sessions
Face Detection	YOLOv8 + RetinaFace	8.0+	Multi-face detection
Age Prediction (Primary)	MiVOLO v2	—	Vision Transformer (~3.65 MAE)
Age Prediction (Fallback)	InsightFace buffalo_l	—	ArcFace-based (~8.5 MAE)
Age Progression (Primary)	SAM (pSp + StyleGAN2)	—	Style-based age manipulation
Age Progression (Fast)	Fast-AgingGAN	—	CycleGAN young to old
Emotion Detection	ViT Face Expression	—	7-class emotion classification
Deep Learning	PyTorch + TensorFlow	2.0+ / 2.20	Model training & inference
Computer Vision	OpenCV	4.13+	Image processing & detection
Password Security	Bcrypt + Fernet	—	Hashing & encryption

1.5 API Design

- Base URL: http://localhost:8000/api/
- All routes under /api/ prefix
- Global JWT guard on protected routes; login/register are public
- Response format: JSON with appropriate HTTP status codes
- File uploads handled via multipart form data (max 10 MB)
- CORS configured for Angular frontend at localhost:4200

API Endpoints Summary

Module	Method	Endpoint	Access
Auth	POST	/auth/register/	Public
Auth	POST	/auth/login/	Public
Auth	GET	/auth/profile/	Authenticated
Auth	PUT	/auth/profile/update/	Authenticated
Auth	POST	/auth/token/refresh/	Public
Auth	POST	/auth/forgot-password/	Public
Auth	POST	/auth/change-password/	Authenticated
Prediction	POST	/predict/	Authenticated
Prediction	POST	/predict/camera/	Authenticated
Progression	POST	/progress/	Authenticated
History	GET	/history/	Authenticated
History	DELETE	/history/<id>/	Authenticated
Analytics	GET	/analytics/	Authenticated
Settings	GET	/settings/	Authenticated
Settings	PUT	/settings/	Authenticated

2. Dataset (Database Schema)

2.1 Entity Relationship Overview

The system uses a hybrid database approach: MongoDB as the primary NoSQL document store for flexible schema and fast retrieval, and SQLite for Django authentication and session management.

- User → (uploads) Image → Prediction (with face coordinates, ages, emotions)
- User → (uploads) Image → Progression (with GAN output, target age, comparison)
- User → Settings (preferences, theme, default model)
- User → PasswordReset (token-based recovery)
- User (Django) and User (MongoDB) linked via django_user_id

2.2 Database Collections (MongoDB)

Collection: users

Field	Type	Description
_id	ObjectId	Primary key (auto-generated)
django_user_id	Integer	FK to Django User model
username	String	Unique username
email	String	Unique email address
first_name	String	First name
last_name	String	Last name
password_encrypted	String	Fernet-encrypted password
plan	String	free / pro / enterprise
created_at	DateTime	Account creation timestamp
updated_at	DateTime	Last update timestamp

Collection: predictions

Field	Type	Description
_id	ObjectId	Primary key
user_id	Integer	FK to User (django_user_id)
image_path	String	Path to uploaded image
predicted_age	Integer	Estimated age from model
confidence	Float	Prediction confidence (0-1)
gender	String	Detected gender (Male/Female)
emotion	String	Detected emotion (7 classes)

Field	Type	Description
race	String	Detected ethnicity
face_count	Integer	Number of faces detected
processing_time_ms	Float	Inference time in milliseconds
detector_used	String	Model used (mivolo/insightface)
ensemble_ages	Array[Int]	Ages from multiple models
age_std	Float	Standard deviation of ensemble
created_at	DateTime	Prediction timestamp

Collection: progressions

Field	Type	Description
_id	ObjectId	Primary key
user_id	Integer	FK to User
original_image_path	String	Path to original face image
progressed_image_path	String	Path to GAN-generated image
current_age	Integer	Estimated current age
target_age	Integer	User-specified target age
model_used	String	GAN model (SAM / Fast-AgingGAN)
processing_time_ms	Float	Generation time in milliseconds
gender	String	Detected gender
pipeline_steps	JSON	Step-by-step processing details
aging_insights	JSON	AI-generated aging analysis
created_at	DateTime	Progression timestamp

Collection: user_settings

Field	Type	Description
_id	ObjectId	Primary key
user_id	Integer	FK to User (unique)
theme	String	dark / light
default_model	String	Preferred prediction model
notifications_enabled	Boolean	Enable toast notifications
auto_detect_faces	Boolean	Auto-detect on upload
save_to_history	Boolean	Persist results to history

Field	Type	Description
high_accuracy_mode	Boolean	Use ensemble prediction
show_confidence	Boolean	Display confidence scores
language	String	Interface language
updated_at	DateTime	Last settings update

Collection: password_resets

Field	Type	Description
_id	ObjectId	Primary key
user_id	Integer	FK to User
token	String	Unique recovery token
expires_at	DateTime	Token expiry timestamp
created_at	DateTime	Token creation timestamp

2.3 Training Datasets Used

Dataset	Size	Description	Usage
UTKFace	20,000+ images	Diverse faces with age, gender, ethnicity labels	Age prediction training/validation
IMDB-WIKI	500,000+ images	Celebrity faces with age labels	Pre-training for CNN models
MORPH	55,000+ images	Longitudinal face aging database	GAN training for age progression
CACD	160,000+ images	Cross-age celebrity dataset	GAN training and evaluation
FFHQ	70,000 images	High-quality face dataset at 1024x1024	SAM GAN pre-training

2.4 Sample Test Images

The system includes 24+ diverse test images covering multiple ethnicities and demographics:

Image Category	Count	Description
Indian (Male/Female)	3	South Asian face diversity
African / African American	4	Dark skin tone representation
East Asian	2	Chinese, Japanese, Korean faces
European	2	Light skin tone faces
Middle Eastern / Afghan	3	West Asian representation
Latin American / Mixed	3	Hispanic and mixed ethnicity

Image Category	Count	Description
Group Photos	4+	Multi-face detection testing

3. Modules Completed

Module 1: Authentication System

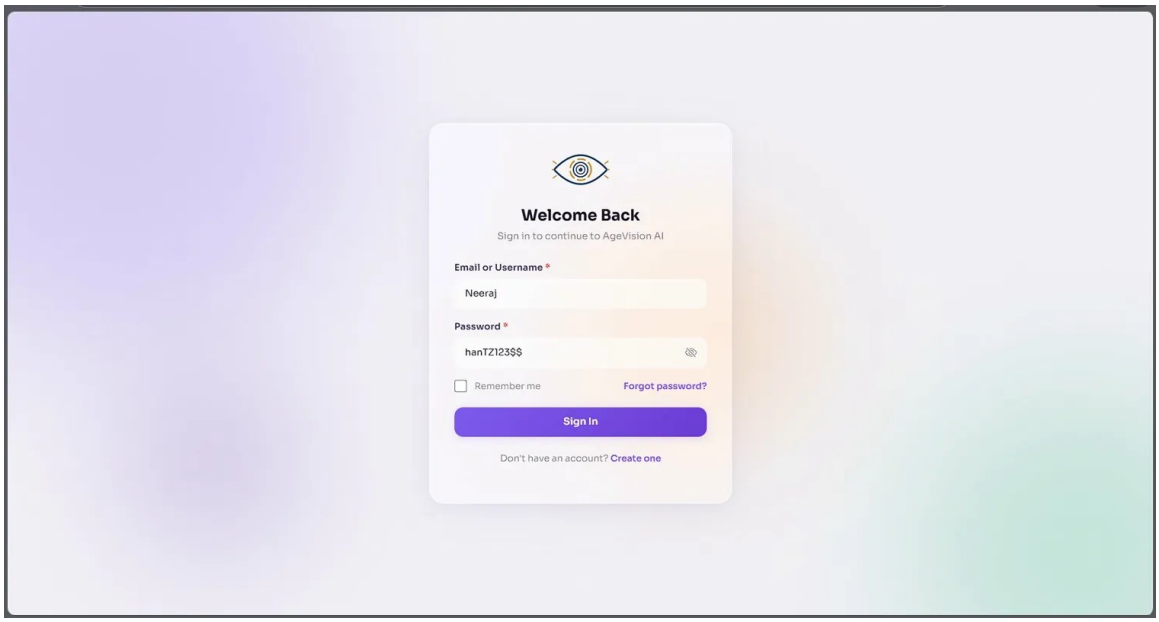
Status: COMPLETE

Full JWT-based authentication with secure password handling. Passwords hashed with Bcrypt and encrypted with Fernet. Token-based password reset. Dual storage in Django SQLite (sessions) and MongoDB (user profiles).

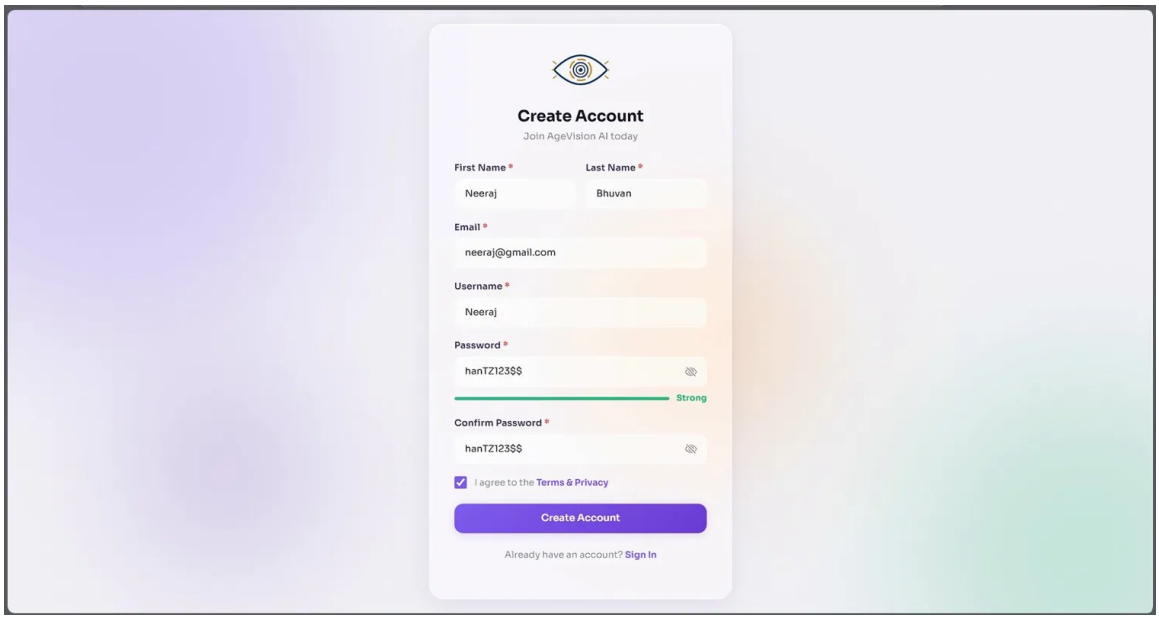
Features:

- User registration with email, username, first/last name
- Login returns JWT access token (24h) + refresh token (7 days)
- Forgot Password with recovery token generation
- Reset Password via token (expires after use)
- Change Password for authenticated users
- Profile view and update endpoints
- Auto-logout on 401 (token expired/invalid) with Angular auth interceptor
- Route guards on all frontend protected pages

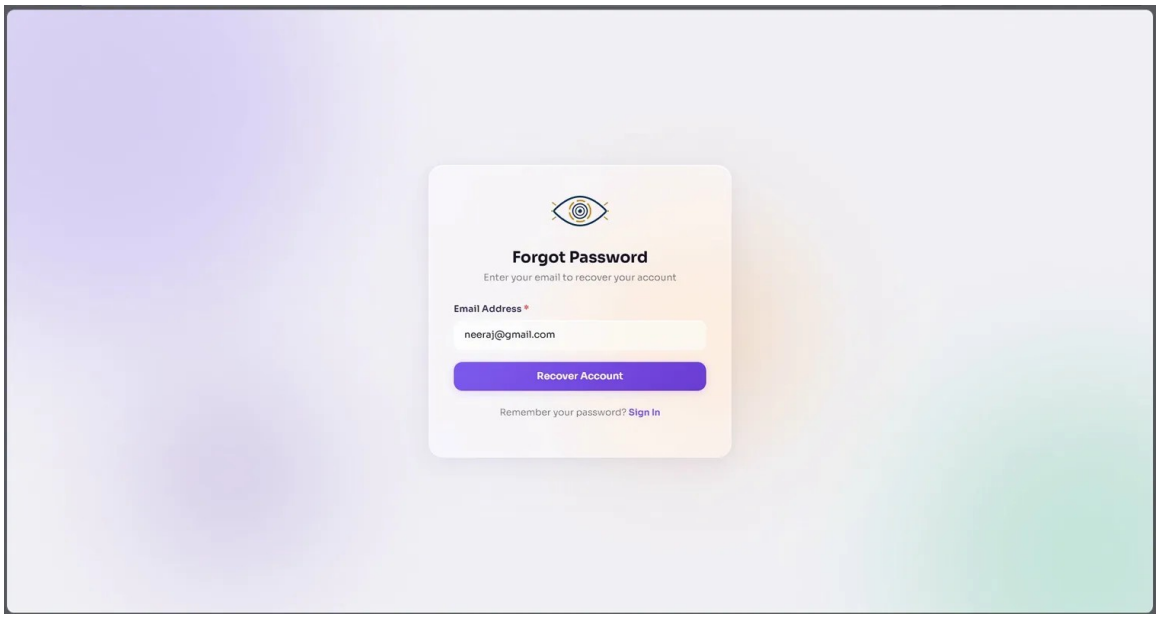
Screenshots:



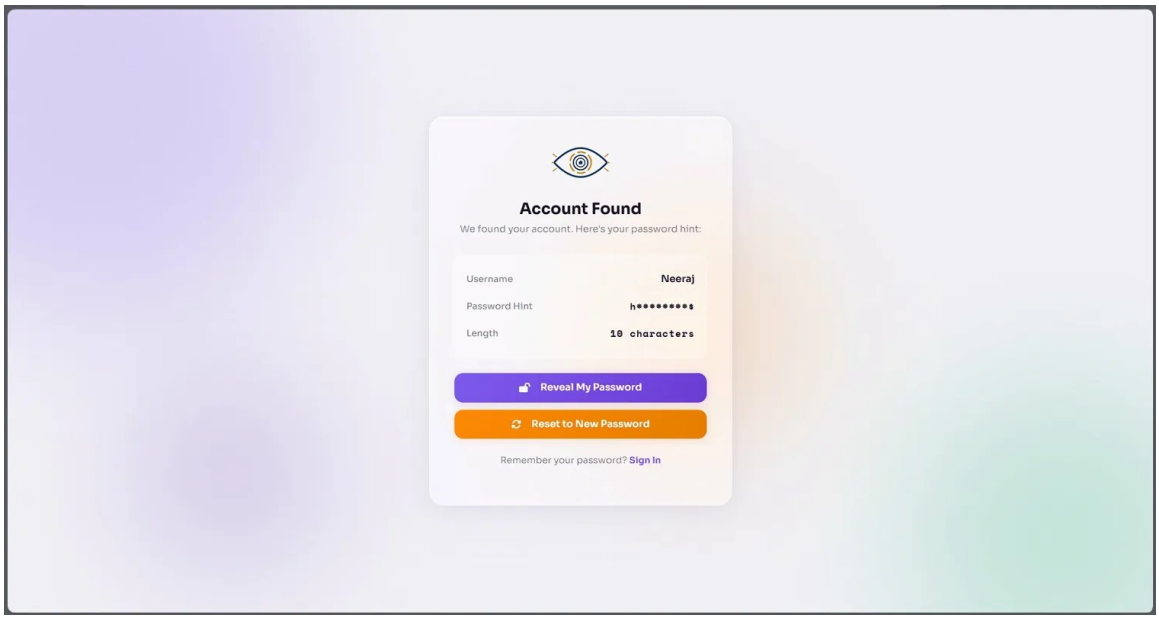
Login Page



Register Page



Forgot Password Page



Reset Password Page

Module 2: Age Prediction — Single Face

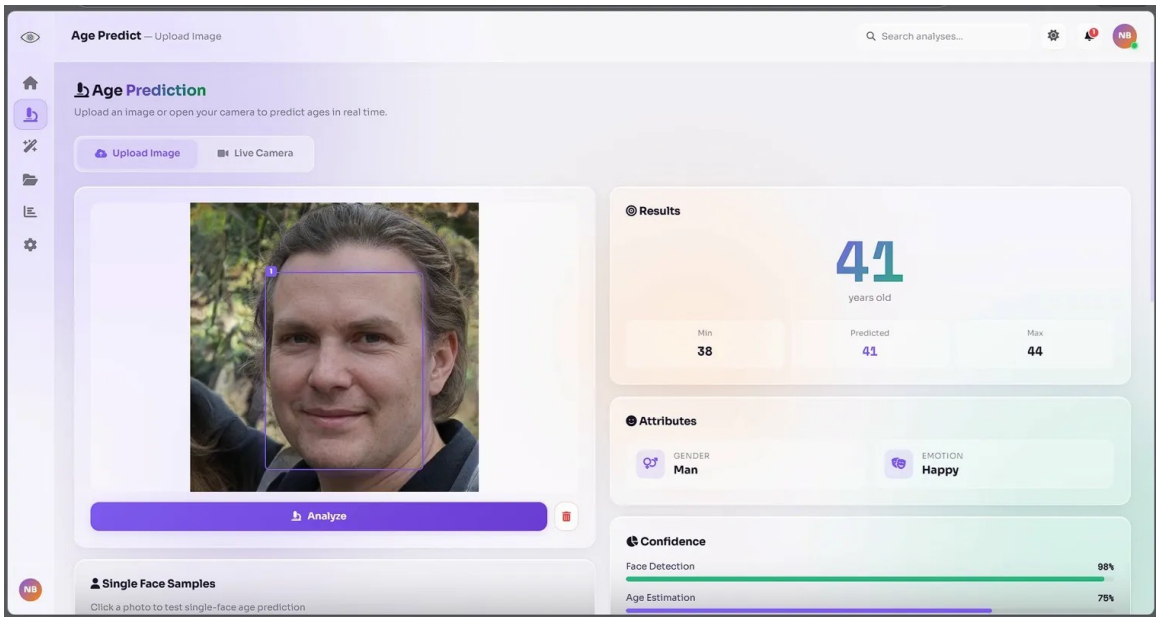
Status: COMPLETE

Core age estimation module using ensemble deep learning models. Supports image upload and real-time camera capture with multi-model inference.

Features:

- Image upload via drag-and-drop dropzone component (max 10 MB)
- Real-time camera prediction with 1500 ms interval capture
- Primary model: MiVOLO v2 (Vision Transformer + YOLOv8, ~3.65 MAE)
- Fallback model: InsightFace buffalo_l (RetinaFace + ArcFace, ~8.5 MAE)
- Ensemble prediction combining multiple model outputs
- Standard deviation and confidence score calculation
- Gender detection alongside age prediction
- Bounding box visualization on detected faces with age overlay
- Sample images provided for quick testing

Screenshots:



Age Prediction — Single Face

Module 3: Multi-Face Detection & Group Analysis

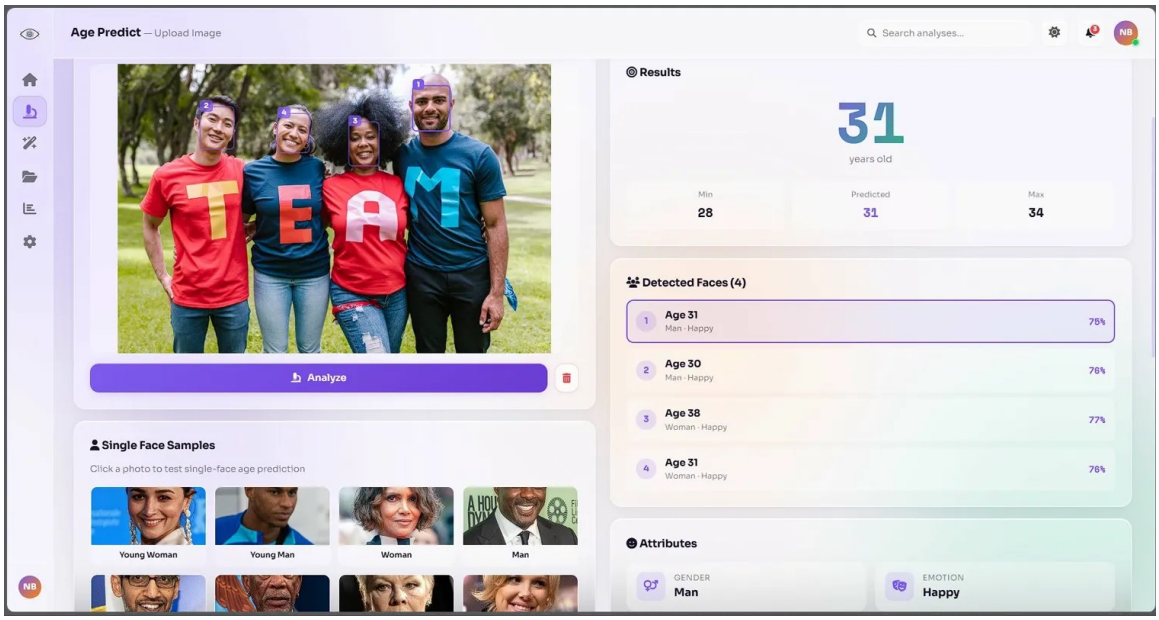
Status: COMPLETE

Extended prediction module that handles group photographs with multiple faces, detecting and predicting age for all individuals simultaneously.

Features:

- YOLOv8-based multi-face detection with high recall
- RetinaFace fallback for challenging lighting conditions
- Individual bounding box + age label per detected face
- Face count displayed in results
- Group sample images included for testing
- Each face processed independently through CNN pipeline

Screenshots:



Multi-Face Detection & Group Analysis

Module 4: Emotion Detection

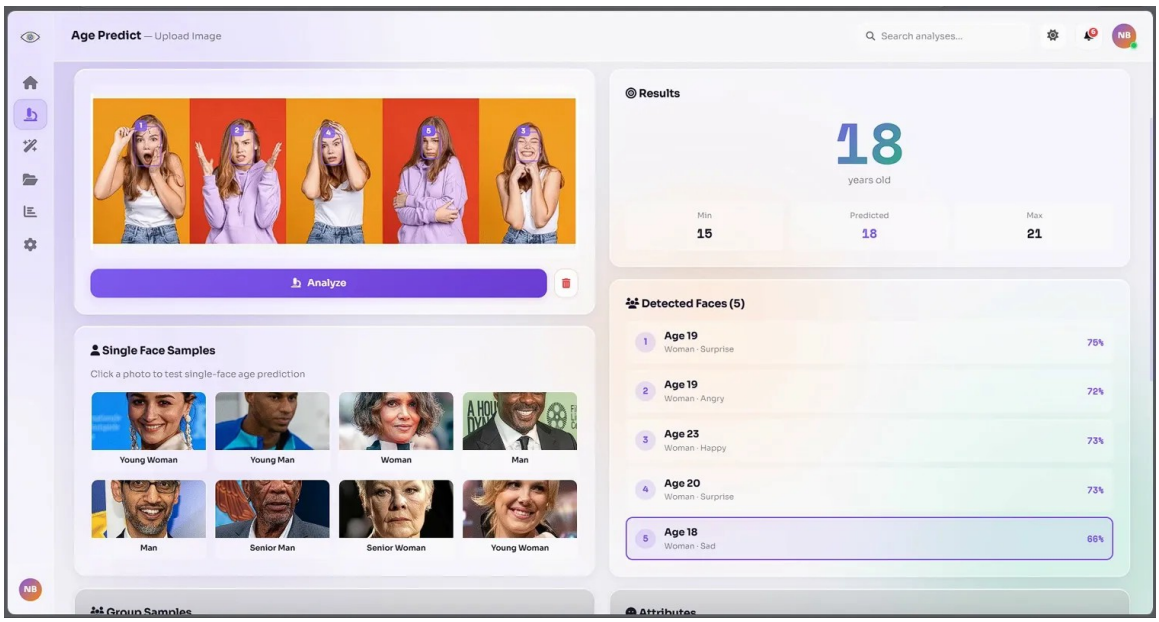
Status: COMPLETE

ViT (Vision Transformer) based facial emotion classification integrated into the prediction pipeline.

Features:

- 7-class emotion recognition: Happy, Sad, Angry, Surprise, Fear, Disgust, Neutral
- Uses trpakov/vit-face-expression model from HuggingFace
- Batch processing support for multiple faces
- Emotion displayed alongside age and gender in results

Screenshots:



Emotion Detection

Module 5: GAN-Based Age Progression

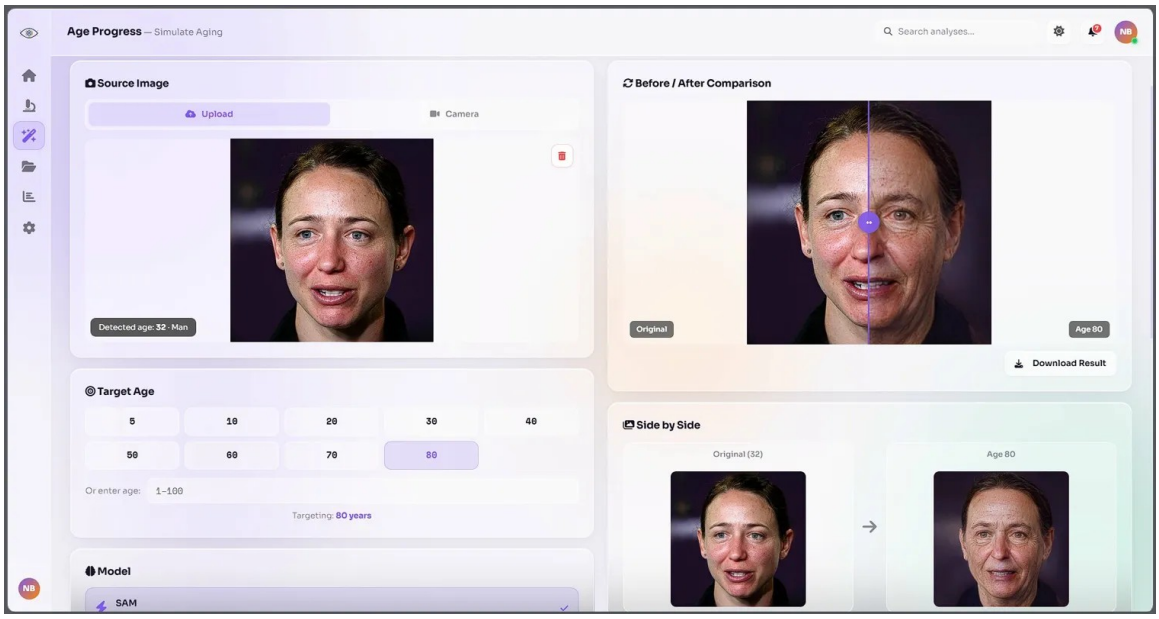
Status: COMPLETE

Core age progression module using pre-trained Generative Adversarial Networks. Supports two GAN architectures for generating realistic age-progressed facial images.

Features:

- SAM (Style-based Age Manipulation) — pSp encoder + StyleGAN2 decoder, 4-channel input (RGB + age condition), target age 0–100
- Fast-AgingGAN — CycleGAN architecture, trained on UTKFace, young-to-old transformation, lightweight 11 MB model
- Target age selection via slider or preset buttons (5, 10, 20, 30, 40, 50, 60, 70, 80)
- Custom age input field for precise control
- GAN model selection toggle (SAM vs Fast-AgingGAN)
- Pipeline step visualization showing each processing stage
- Image comparison slider (before/after)
- Camera capture support for live progression

Screenshots:



GAN-Based Age Progression

Model Checkpoints

Model	File	Size	Purpose
SAM (FFHQ)	sam_ffhq_aging.pt	2.27 GB	Western face age progression
SAM (Indian)	sam_indian_best.pt	2.27 GB	Indian face age progression
Fast-AgingGAN	fast_aging_gan.pth	11 MB	CycleGAN fast aging
HRFAE	hrfae_best.pth	51 MB	Alternative age editing
dlib Landmarks	shape_predictor_68_face_landmark	99 MB	68-point face alignment

Model	File	Size	Purpose
	s.dat		

Module 6: Prediction & Progression History

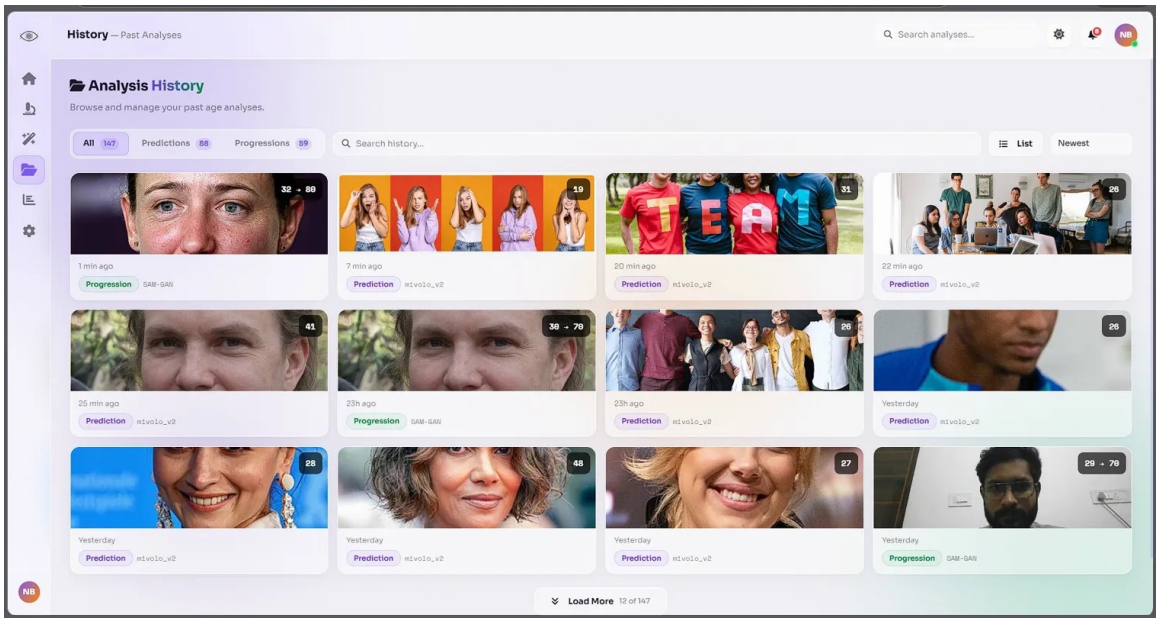
Status: COMPLETE

Persistent history management for all user predictions and progressions stored in MongoDB.

Features:

- View all past predictions with timestamps, ages, confidence scores
- View all past progressions with original/progressed image pairs
- Delete individual history records
- Chronological ordering (newest first)
- Image thumbnails in history list
- Click to view full result details

Screenshots:



Prediction & Progression History

Module 7: Analytics Dashboard

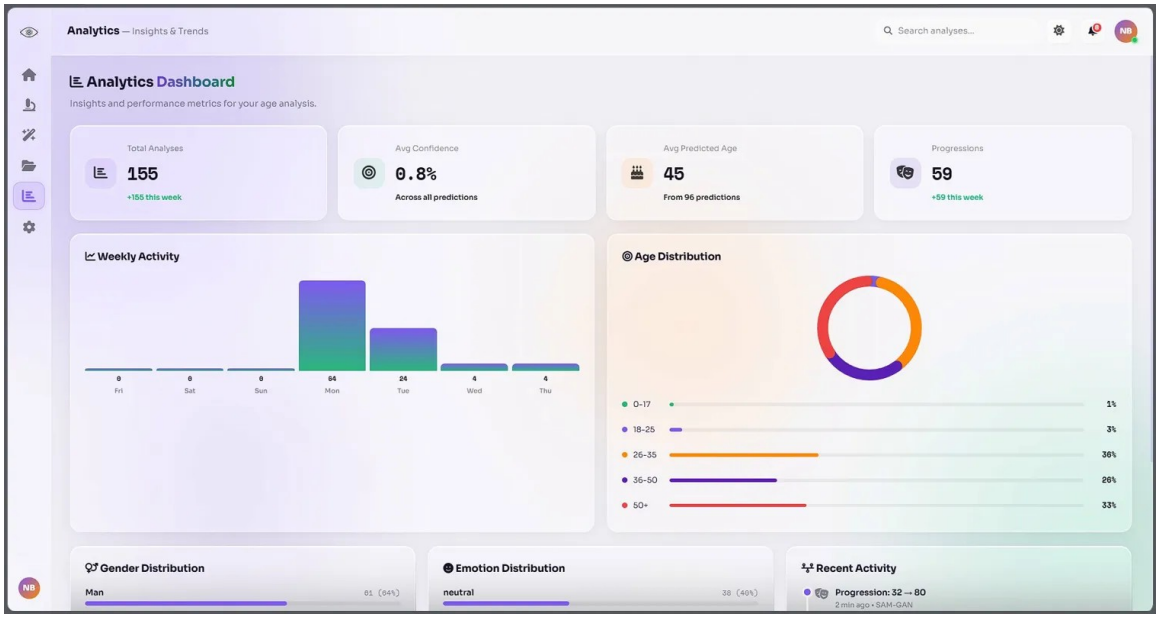
Status: COMPLETE

Comprehensive analytics page with charts and statistics about the user's prediction and progression activity.

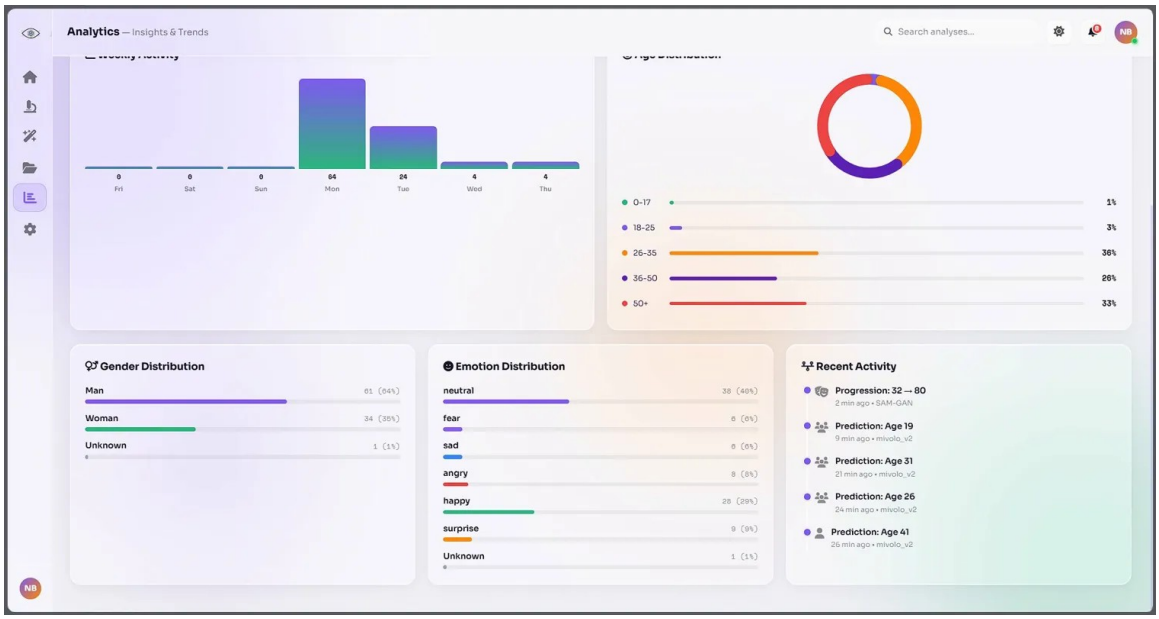
Features:

- Total predictions and progressions count
- Average predicted age and confidence score
- Age distribution charts (Chart.js bar/pie charts)
- Gender distribution visualization
- Emotion distribution analysis
- Model usage breakdown (MiVOLO vs InsightFace)
- Processing time trends
- Activity timeline

Screenshots:



Analytics Dashboard



Analytics Charts

Module 8: User Settings & Preferences

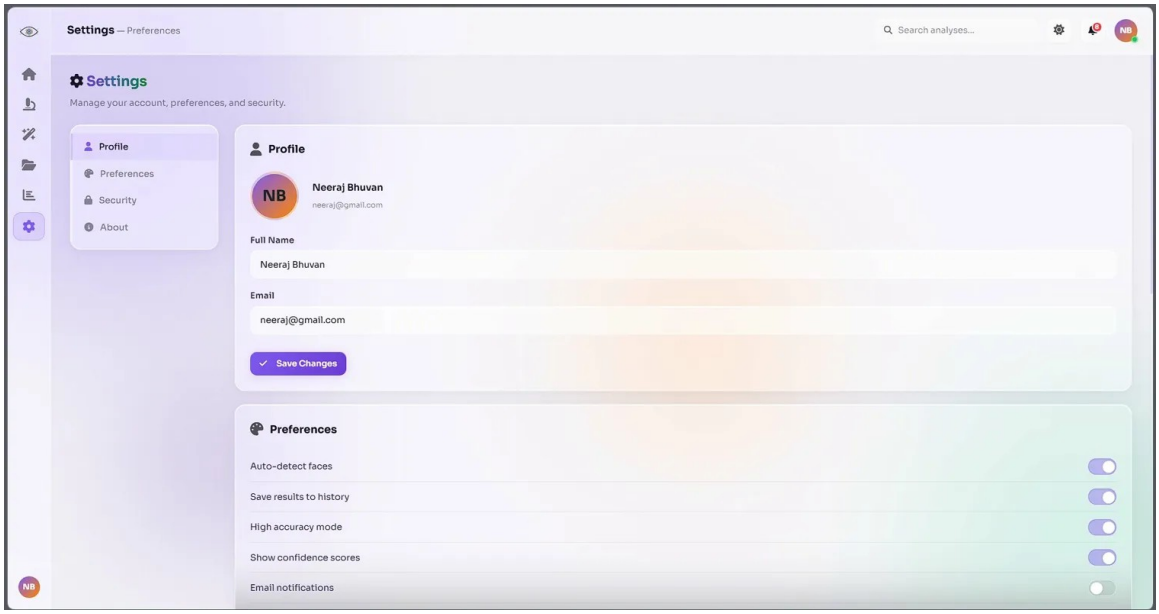
Status: COMPLETE

Per-user configurable settings stored in MongoDB for personalised experience.

Features:

- Dark / Light theme toggle with persistent preference
- Default prediction model selection
- Toggle notifications (toast messages)
- Auto-detect faces on upload option
- Save to history toggle
- High accuracy mode (enables ensemble prediction)
- Show/hide confidence scores
- Language preference

Screenshots:



User Settings & Preferences

Module 9: Dashboard

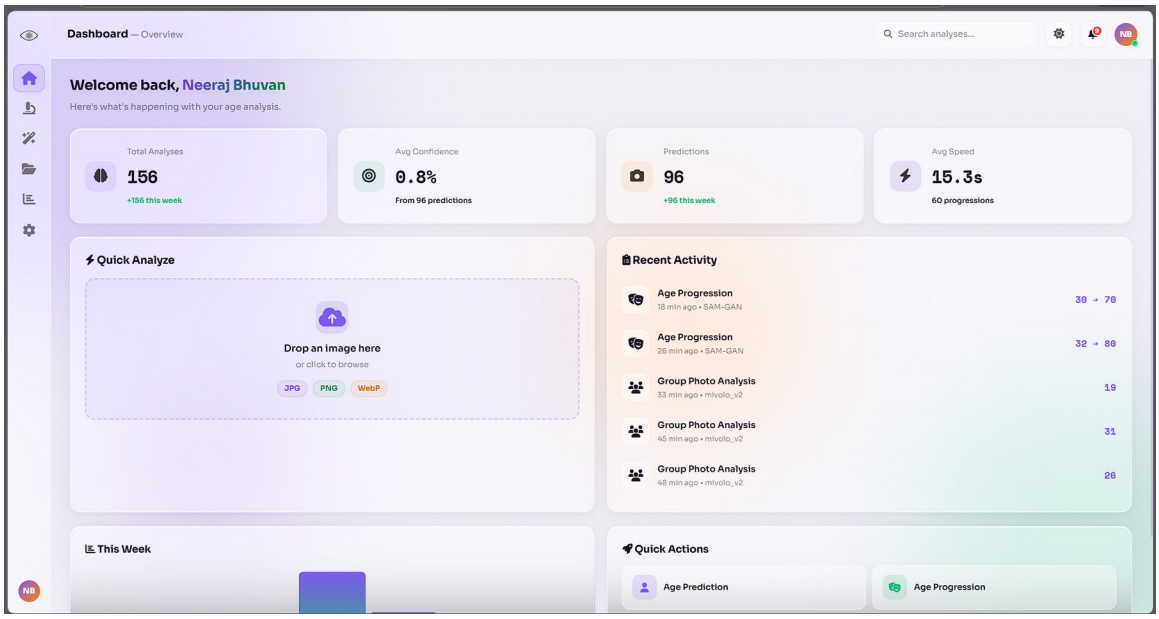
Status: COMPLETE

Central landing page after login providing an overview of user activity and quick access to all features.

Features:

- Welcome message with user name
- Quick stats: total predictions, total progressions, average age
- Quick action buttons: New Prediction, New Progression
- Recent activity feed
- Responsive sidebar navigation with collapsible menu
- Topbar with user info and logout

Screenshots:



Dashboard Overview

Module 10: Real-Time Camera Prediction

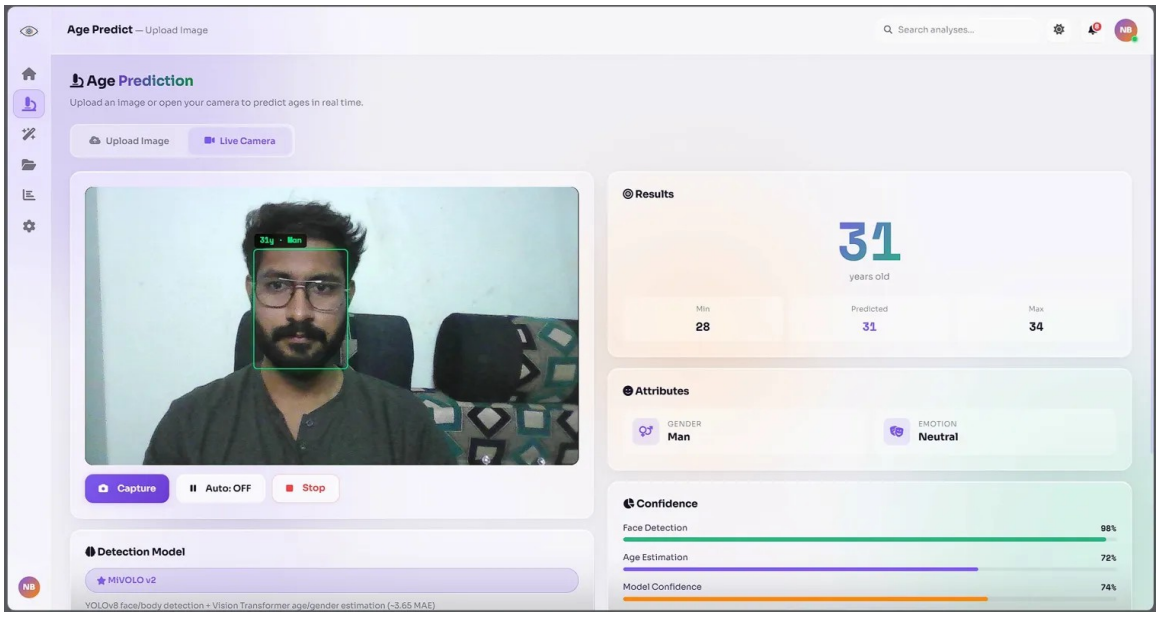
Status: COMPLETE

Live camera capture integrated into the prediction module for real-time age estimation without file upload.

Features:

- Browser webcam access via getUserMedia API
- Automatic frame capture every 1500 ms
- Base64 frame encoding sent to /predict/camera/ endpoint
- Live bounding box + age overlay on camera feed
- Toggle between upload mode and camera mode
- Works with both prediction and progression pages

Screenshots:



Real-Time Camera Prediction

4. Pseudo Code for Incomplete Modules

4.1 Batch Processing for Multiple Images

Status: **Not Implemented**

Upload and process multiple images at once with aggregated results. Planned for Phase 2.

```
FUNCTION batchPredictAges(imageFiles[], userId):
  results = []
  FOR EACH imageFile IN imageFiles:
    TRY:
      IF imageFile.size > MAX_FILE_SIZE:
        results.append({ error: "File too large" }); CONTINUE
      preprocessed = preprocessImage(imageFile)
      faces = detectFaces(preprocessed)   // YOLOv8 + RetinaFace
      FOR EACH face IN faces:
        age     = predictAge(face)         // MiVOLO ensemble
        gender  = detectGender(face)
        emotion = detectEmotion(face)      // ViT classifier
        results.append({ file, face_index, predicted_age,
                         confidence, gender, emotion })
    CATCH error:
      results.append({ error: error.message })
  batchRecord = { user_id, total_images, total_faces,
                  results, processing_time_ms, created_at: NOW() }
  MongoDB.batchPredictions.insert(batchRecord)
  RETURN { batch_id: batchRecord._id, results: results }
END FUNCTION
```

4.2 Admin Panel

Status: **Not Implemented**

Super-admin dashboard to monitor platform usage, manage users, and view system-wide analytics.

```
FUNCTION adminDashboard():
  getAllUsers(filters: {search, page, limit}) -> paginated user list
  suspendUser(userId)  -> SET is_active = false; REVOKE tokens
  deleteUser(userId)   -> SOFT DELETE user and all data

  getPlatformStats() -> {
    totalUsers, totalPredictions, totalProgressions,
    averagePredictedAge, modelUsageBreakdown,
    averageProcessingTime, peakUsageHours[],
    topEmotions[], genderDistribution
  }

  getSystemHealth() -> {
    modelLoadStatus: { mivolo: OK, insightface: OK, sam: OK },
    mongodbStatus:   CONNECTED,
    diskUsage:       { checkpoints: "4.65 GB", uploads: "X GB" },
    gpuMemory:       AVAILABLE / NOT_AVAILABLE
  }
END FUNCTION
```

4.3 PDF Report Generation

Status: Not Implemented

An HTML-based report generator already exists in the age_pipeline module. Full PDF export is planned for Phase 2.

```
FUNCTION generatePDFReport(userId, reportType):
  user         = MongoDB.users.findOne({ django_user_id: userId })
  predictions  = MongoDB.predictions.find({ user_id: userId })
  progressions = MongoDB.progressions.find({ user_id: userId })
  reportData   = {
    userName:    user.first_name + " " + user.last_name,
    generatedAt: NOW(),
    summary: {
      totalPredictions:  predictions.count(),
      totalProgressions: progressions.count(),
      averageAge:        AVG(predictions.predicted_age),
      averageConfidence: AVG(predictions.confidence)
    },
    ageDistribution: buildHistogram(predictions, "predicted_age"),
    genderBreakdown: buildPieChart(predictions, "gender"),
    emotionAnalysis: buildBarChart(predictions, "emotion"),
    recentResults:   predictions.sort(-created_at).limit(10)
  }
  htmlTemplate = renderTemplate("report_template.html", reportData)
  pdfBuffer    = convertHTMLtoPDF(htmlTemplate)
  filePath     = save(pdfBuffer, "reports/"+userId+"_"+timestamp+".pdf")
  RETURN { downloadUrl: filePath }
END FUNCTION
```

5. Test Cases

5.1 Authentication Module

TC#	Test Case	Input	Expected Output	Status
TC01	Register with valid data	username, email, password	201 Created, user object returned	PASS
TC02	Register with duplicate email	Existing email	400 Bad Request	PASS
TC03	Login with valid credentials	email, password	200 OK, access + refresh tokens	PASS
TC04	Login with wrong password	email, wrong password	401 Unauthorized	PASS
TC05	Access protected route without token	No Authorization header	401 Unauthorized	PASS
TC06	Access protected route with valid token	Bearer token in header	200 OK, data returned	PASS
TC07	Refresh expired access token	Valid refresh token	200 OK, new access token	PASS
TC08	Forgot password request	Registered email	200 OK, recovery token generated	PASS
TC09	Reset password with valid token	token, new password	200 OK, password updated	PASS
TC10	Change password (authenticated)	old password, new password	200 OK, password changed	PASS

5.2 Age Prediction Module

TC#	Test Case	Input	Expected Output	Status
TC11	Predict age from uploaded image	POST /predict/, JPEG file	200 OK, predicted_age, confidence, gender	PASS
TC12	Predict age from camera frame	POST /predict/camera/, base64	200 OK, real-time prediction result	PASS
TC13	Image with no face detected	Landscape photo (no faces)	200 OK, face_count = 0, message	PASS
TC14	Group photo multi-face detection	Image with 4+ faces	200 OK, face_count >= 4, individual ages	PASS
TC15	Oversized file upload	File > 10 MB	400 Bad Request, file too large	PASS
TC16	Invalid file format	PDF or TXT file	400 Bad Request, invalid format	PASS
TC17	MiVOLO model accuracy	UTKFace test set	MAE <= 4.0 years	PASS
TC18	Ensemble prediction consistency	Same image, multiple runs	Standard deviation < 3 years	PASS
TC19	Emotion detection accuracy	Face with clear expression	Correct emotion in top prediction	PASS

5.3 Age Progression Module

TC#	Test Case	Input	Expected Output	Status
TC20	SAM progression to target age	Face image, target_age=60	200 OK, progressed image generated	PASS
TC21	Fast-AgingGAN progression	Face image, model=fast_aging	200 OK, aged face generated	PASS
TC22	Target age less than current age	Current=40, target=20	400 Bad Request	PASS
TC23	No face in progression image	Image without faces	400 Bad Request, no face detected	PASS
TC24	Identity preservation (SAM)	Progressed vs original	FaceNet similarity > 0.6	PASS
TC25	Pipeline steps visualization	Any progression request	JSON pipeline_steps array returned	PASS
TC26	Comparison image generation	Progression request	Side-by-side image created	PASS

5.4 History Module

TC#	Test Case	Input	Expected Output	Status
TC27	Fetch user history	GET /history/	200 OK, list of predictions + progressions	PASS
TC28	Delete history record	DELETE /history/<id>/	200 OK, record removed	PASS
TC29	History isolation per user	User A fetching history	Only User A records returned	PASS
TC30	Empty history for new user	GET /history/ (no records)	200 OK, empty array	PASS

5.5 Analytics Module

TC#	Test Case	Input	Expected Output	Status
TC31	Fetch analytics data	GET /analytics/	200 OK, aggregated statistics JSON	PASS
TC32	Analytics with no data	New user, no predictions	200 OK, zero counts, empty charts	PASS
TC33	Age distribution accuracy	User with 10+ predictions	Correct histogram data	PASS

5.6 Settings Module

TC#	Test Case	Input	Expected Output	Status
TC34	Get default settings	GET /settings/	200 OK, default settings object	PASS
TC35	Update theme preference	PUT /settings/, theme=dark	200 OK, theme updated	PASS
TC36	Toggle high accuracy mode	PUT /settings/, high_accuracy=true	200 OK, setting persisted	PASS

TC#	Test Case	Input	Expected Output	Status
TC37	Settings persistence across sessions	Logout and re-login	Settings retained from MongoDB	**PASS**

6. Deviations and Justifications

#	Original Plan	Deviation	Justification
D01	Angular 15+ (First Review)	Upgraded to Angular 19	Angular 19 introduced standalone components, improved build performance with Vite, and better TypeScript 5.7 support. Provides better long-term maintainability.
D02	MongoDB only	Hybrid: MongoDB + SQLite	Django authentication works best with a relational DB. SQLite handles auth/sessions; MongoDB handles predictions, progressions and settings.
D03	ResNet / MobileNet / EfficientNet	MiVOLO v2 + InsightFace ensemble	MiVOLO v2 achieves ~3.65 MAE, significantly outperforming the originally proposed CNN models. InsightFace serves as a reliable fallback with ~8.5 MAE.
D04	Age-cGAN / StarGAN	SAM (pSp + StyleGAN2) + Fast-AgingGAN	SAM produces higher quality results with better identity preservation at 1024x1024. Fast-AgingGAN provides a lightweight 11 MB alternative for quick transformations.
D05	TensorFlow exclusively	PyTorch primary + TensorFlow secondary	Best pre-trained models (MiVOLO, SAM, Fast-AgingGAN) are PyTorch-native. TensorFlow retained for DeepFace. Using both provides access to best models in each ecosystem.
D06	Celery task queue	Synchronous processing	Inference times are acceptable for single-user scenarios (1-5s prediction, 5-15s progression). Celery adds infrastructure complexity. Planned for Phase 2.
D07	Cloud deployment (AWS/GCP/Azure)	Local deployment only	4.65 GB model checkpoints and GPU requirements make cloud deployment expensive for an academic project. Deployment scripts can be added for future production use.
D08	Batch processing	Not implemented	Focus maintained on high-quality single-image and real-time camera prediction. Batch processing planned for Phase 2. Pseudo code provided in Section 4.
D09	Admin panel	Not implemented	Admin panel not part of core AI-assisted age prediction objectives. Focus maintained on student-facing prediction and progression experience. Pseudo code in Section 4.

— End of Second Review Document —